

The Cursor Dev Loop: From Business Concept to Functional Code

The "Document-First" Philosophy

In traditional software development, a developer often carries the entire project state in their head. However, when working with AI, we encounter the "Memory Problem": AI models have a limited context window. If you rely solely on a chat interface, the AI will eventually "forget" earlier decisions or requirements as the conversation grows.

The Cursor Dev Loop solves this by externalizing the project state into Markdown files. Instead of keeping the plan in the chat history, we keep it in the codebase itself. These documents serve as the AI's "long-term memory," ensuring that every coding session starts with a perfect understanding of the goals and constraints.

1. Plans are cheap, but code is expensive. Dedicating the first four steps of the loop to planning ensures that the implementation phase is efficient and predictable.
2. Implementation is an exercise in assembly, not invention. When the blueprint is sufficiently detailed, the AI acts as a precision tool for construction rather than a generator of guesses.

This shift from manual coding to orchestrated assembly is enforced by a technical framework of rules and modes that define the boundaries of the AI's behavior.

The Engine: Cursor Rules and Modes

The heart of this workflow is a series of .mdc files known as Cursor Rules. These files guide the AI through specific phases and ensure it adheres to your project's unique technical requirements.

The Developer's Toolkit

Rule File Purpose Activates On

prd-creation.mdc PRD generation from BRD or human prompts. **/docs//.md**

implementation-plan.mdc Architecture decisions and phase breakdown. **/docs//.md**

implementation-steps.mdc Decomposing the plan into isolated phase files. **/docs//.md**

implement-step.mdc Executing a single phase with strict verification. ****/*.ts, **/*.tsx, /docs//.md**

code-style.mdc Enforcing naming, structure, and hook patterns. ****/*.js, **/*.jsx, **/*.ts, **/*.tsx**

tech-stack.mdc Defining stack conventions (Supabase, TanStack, Zod). ****/*.ts, **/*.tsx**

shadcn-ui.mdc Guiding consistent UI component patterns. **/components//.tsx, /app//.tsx**

github-workflow.mdc Managing commit conventions and branching. ***/**

Auto-activation vs. Explicit Referencing (@)

Cursor can trigger rules automatically based on file globs, but a senior developer understands when to intervene:

- Auto-activation: Rules fire based on the file patterns listed above. Use this for general style and tech stack enforcement while the AI is writing code.
- Explicit Referencing (@): The developer manually mentions a rule (e.g., @implement-step.mdc) in the prompt.
 - Why use it? This is the Best Practice for process-heavy tasks. It provides guaranteed full context, ensuring the AI sees the complete rule content—templates, checklists, and code examples—rather than a truncated summary. It also signals clear intent for the AI to follow a specific multi-step workflow.

These rules provide the rigid guardrails necessary to transform a fluid business concept into a structured technical requirement.

Phase 1: Defining the North Star (BRD & PRD)

The journey begins with the Business Requirements Document (BRD)—the raw stakeholder input. The first major transformation is turning this raw input into a Product Requirements Document (PRD) using Step 2 of the loop.

Moving from a BRD to a structured PRD is critical because it translates human intent into a format optimized for AI reasoning. For this reasoning-heavy phase (Steps 1–4), you must utilize high-intelligence models (Claude 3.5 Sonnet or Opus). The PRD serves as an "anchor" that prevents logic errors by defining the "What" and "Why" before the "How."

Three Pillars of the PRD

1. Data Model: Unlike traditional PRDs, this includes TypeScript interfaces and Zod schemas from the start, providing technical clarity the AI can build upon immediately.
2. Acceptance Criteria: Clearly defined "done" states for every feature to prevent scope creep.
3. Context Preservation: The PRD is self-contained. The goal is that any future session can start by reading this single file to understand the entire feature scope.

Checklist: PRD Output

- ☐ Problem Statement: Clear definition of the user pain point.
- ☐ User Stories: Detailed "As a... I want... So that..." narratives.
- ☐ Data Model: Technical type definitions and schema examples.
- ☐ UI/UX & API Requirements: Layout constraints and endpoint signatures.
- ☐ Out of Scope: Explicit boundaries to focus the AI's attention.

With the "What" solidified, we move to Step 3: defining the technical architecture.

Phase 2: The Architectural Blueprint (Implementation Plan)

The Implementation Plan forces a review of architecture decisions before a line of functional code is written. It is far more efficient to refactor a bullet point in a plan than a complex database schema.

The `implementation-plan.mdc` rule primes the AI with project-specific patterns, including:

- Server Actions: Specific Next.js use server patterns for mutations.
- TanStack Query: Standardized hooks for data fetching and caching.
- SQL Migrations: Database templates that include Row Level Security (RLS) policies, indexes, and triggers.

The most critical artifact here is the Phase Breakdown Table. This table decomposes the massive architectural goal into small, manageable phases with estimated complexity. This roadmap is the foundation for the "State Machine" that will track our progress.

Phase 3: Creating the State Machine (Step Files & Index)

In Step 4, we decompose the plan into isolated, self-contained phase files. This creates the project's State Machine, orchestrated by the `00-index.md` file.

The Heart of the System: `00-index.md`

The index file is the source of truth for the project's status. It tracks progress through a status table and a "Current Context" prose section. This prose section is vital for resilience; it describes the current state of the world so that any developer (or AI) can resume work instantly.

Status Meaning

Pending Task is scheduled but not yet started.

In Progress Task is currently being executed.

Complete Task is finished, verified, and committed.

Blocked Task cannot proceed due to prerequisites or external factors.

The Power of Isolation: By creating "Self-Contained Phases," we isolate the AI's attention. Each phase file (e.g., `01-schemas.md`) contains enough code and context to be implemented without the AI needing to recall previous chat history. This isolation is the primary defense against AI hallucination.

Once the state machine is initialized, we move from the high-reasoning world of planning to the disciplined world of execution.

Phase 4: The Execution Loop (Implement, Verify, Commit)

Execution is governed by the `implement-step.mdc` rule, which follows a strict 7-step sequence:

1. Load Context: Read [00-index.md](#) and the specific phase file.
2. Safety Check: Verify a clean git status to prevent losing work.
3. Update Status: Mark the phase as "In Progress" in the index.
4. Execute: Follow the plan, checking off items as they are built.
5. Verify: Run the mandatory Verification Gate.
6. Update Documentation: Mark phase as "Complete" and record Implementation Notes (capturing any architectural drift or deviations).
7. Commit: Save the work using standard conventions.

The Verification Gate & Pre-Commit Checklist

The `npm run build` command is the minimum requirement, but a true senior-level workflow requires a rigorous Pre-Commit Checklist:

- ☐ Code compiles without errors (`npm run build`).
- ☐ No `console.log` statements in production code.
- ☐ No secrets or API keys in the codebase.
- ☐ TypeScript types are correct (no usage of any).
- ☐ Feature behavior matches the phase requirements.

Commit Conventions

Type Usage

`feat` New feature or phase completion.

`fix` Bug fix.

`refactor` Code change that neither fixes a bug nor adds a feature.

`docs` Documentation only changes.

`test` Adding missing tests or correcting existing tests.

`chore` Changes to the build process or auxiliary tools.

Even the most disciplined developer needs to step away; the loop is designed to make returning to work seamless.

Maintaining Continuity: Session Management & Context Recovery

The true strength of the Cursor Dev Loop is its resilience. If the AI loses its way or you start a new session, the documentation provides the path back.

The Golden Rule for Context Overload: Drag only one phase file at a time into the AI's context window. The isolation of phases is also the isolation of AI attention. Too many files lead to focus drift and

hallucination.

Resuming Work

When opening a fresh session, provide the AI with the exact state and process instructions: "Review `@.cursor/rules/implement-step.mdc` and `@docs/[feature-name]/00-index.md`. What is the next step?"

Mid-Phase Interruptions

If you are interrupted mid-task, follow this 3-step recovery checklist:

1. Ask the AI to update the current phase file with notes on what is done and what remains.
2. Update the `00-index.md` "Current Context" prose to reflect the mid-task state.
3. Commit your work-in-progress if it passes the build verification.

Summary: Why the Loop Works

The Cursor Dev Loop transforms the developer from a manual "coder" into an orchestrator of AI-driven assembly. This workflow succeeds for four critical reasons:

- Context Survival: State is preserved in persistent Markdown files, not volatile chat history.
- Consistency: Rules fire automatically to ensure naming, security (RLS), and architectural patterns are followed.
- Risk Reduction: Small, verifiable increments (phases) and mandatory build gates catch errors before they compound.
- Efficiency: Because planning is prioritized (Steps 1–4), the AI generates highly accurate code on the first attempt during implementation.

By externalizing memory and automating standards, you enable a development environment where progress is never lost, and the project state is always clear. This is the future of resilient, AI-assisted engineering.